TD3 — Working as a team on GitHub

Git, Python Environments & Coding Best Practices

MSA-DATB07-03 — 1 h 30 practical

Learning outcome. Clone a team repository, split the work with Issues, push a branch, open a pull request, review a teammate's pull request and merge it, resolve a conflict in a pull request, and see branch protection at work.

Tip — how this TD works

You work in teams of 3 or 4, in one GitHub repository per team: a pull request needs a teammate to review it. As in TD2: **guided** steps give the commands, **Your turn** steps ask you to adapt them, and ★ marks the key questions, the kind of reasoning asked in the quiz. Everything of today goes into a folder `td3`, next to `td2`. Files are created and edited in VS Code (method 1 of TD2); the terminal commands of method 2 work too. This TD is not graded.

Tip — reminder from TD2: the loop, and branches

Today everything goes through these commands, now with teammates:

- **save a change**: edit and save the file (VS Code) → `git status` (what changed?) → `git add <file>` (put it in the next commit) → `git commit -m "message"` (save it in the history) → `git log --oneline`
- **a branch**: `git switch -c <name>` (create it and go to it), `git switch main` (back on `main`), `git merge <branch>` (bring a branch into the one you are on)
- **a conflict**: keep the right lines, delete the three marker lines, save → `git add` → `git commit`

New today: `git push` sends your commits to GitHub, `git pull` brings back those of your teammates.

Part A — Your team and its repository *(20 min)*

Exercise 1 — Form the team, create the repository

Form teams of 3 or 4; the instructor gives each team a number N. One member is the **owner**: they create the repository, the others are **collaborators**. Everybody needs the GitHub account of TD2, signed in on https://github.com.

1. **Owner only.** Open https://github.com/msa-datb07-git/team-project-template, click *Use this template* → *Create a new repository*. Owner: your account; name: `msa-datb07-teamN`; visibility: **Public**.
2. **Owner only.** *Settings* → *Collaborators* → *Add people*: add each teammate with their GitHub user name. **Teammates**: accept the invitation (e-mail, or the bell icon on GitHub).
3. **Owner only.** Protect `main`: *Settings* → *Branches* → *Add classic branch protection rule*. Branch name pattern `main`; tick *Require a pull request before merging* and *Require approvals* (1); at the bottom, tick *Do not allow bypassing the above settings* (otherwise the owner could still push on `main`). Save.

Exercise 2 — Clone it

Guided, everybody. On the team repository, the green *Code* button gives the address (*HTTPS* tab). Predict, run, explain:

```
git config --global pull.rebase false      # once per computer: git pull = download + merge
mkdir -p ~/Documents/courses/git/td3       # your course folder, next to td2
cd ~/Documents/courses/git/td3
```

```
git clone https://github.com/<owner>/msa-datb07-team<N>.git
cd msa-datb07-team<N>
git remote -v                                  # (1)
git log --oneline                              # (2)
python -m pip install pytest                   # macOS: python3 -m pip
python -m pytest                               # (3)
```

1. In (1), what is `origin`? In (2), how many commits does the repository have?
2. In (3), how many tests pass?
3. Open the folder `msa-datb07-team<N>` in VS Code (*File* → *Open Folder*) and read `TASKS.md`: what is the task of each member of your team today?

Part B — Split the work with Issues *(10 min)*

Exercise 3 — One Issue per task

Where you should be: `pwd` ends with `courses/git/td3/msa-datb07-team<N>`
If not: `cd ~/Documents/courses/git/td3/msa-datb07-team<N>`

An **Issue** is a card in the to-do list of the repository: everyone sees what must be done, and who does it.

1. Share tasks T1 to T4 of `TASKS.md`: one per member. T1 and T2 go to two different people, T3 and T4 too. Teams of 3: one member also takes T4, after their first task.
2. **Each member**, on GitHub: tab *Issues* → *New issue*. Title: the title of your task (e.g. `T1 --- add average_order_value`). On the right, *Assignees* → *assign yourself*. *Create*. Note the number GitHub gives it: `#1`, `#2`...

Your turn Open the *Issues* tab: who does what in your team? Which Issue number is yours?

Part C — One task each, through pull requests *(45 min)*

Exercise 4 — Branch, commit, push, pull request

Where you should be: `pwd` ends with `courses/git/td3/msa-datb07-team<N>`
If not: `cd ~/Documents/courses/git/td3/msa-datb07-team<N>`

Guided — the author. Example with T1; use the branch name and the message of your task:

```
git switch main
git pull                                       # start from the latest main
git switch -c feat/average-order-value         # a branch for your task
```

Do the task in VS Code (code, test, README), save. Then:

```
python -m pytest                               # the tests must pass
git add -A                                     # -A = all the changes
git commit -m "feat(metrics): add average_order_value"
git push -u origin feat/average-order-value    # -u: remember where this branch goes
```

The first push may ask you to sign in to GitHub: if it does not work, see the annex at the end of this TD. Git prints a link: open it, or click *Compare & pull request* on GitHub. Fill in the template: *What*, *Why* with `Closes #1` (your Issue number: merging the pull request will close the Issue). Choose a teammate in *Reviewers*. *Create pull request*.

Guided — the reviewer. Open the pull request, tab *Files changed*. Click on a line to comment: a question, a suggestion, a possible bug. Then *Review changes* → *Approve*, or *Request changes* if something must change first (the author fixes it with a new commit on the same branch and pushes again: the pull request updates itself).

Guided — after the approval. The author clicks *Merge pull request*, then *Delete branch*. Everybody brings `main` up to date:

```
git switch main
git pull
```

Exercise 5 — The planned conflicts

Where you should be: `pwd` ends with `courses/git/td3/msa-datb07-team<N>`
If not: `cd ~/Documents/courses/git/td3/msa-datb07-team<N>`

T1 and T2 change the same lines (`__init__.py`, the *Features* list of the README); T3 and T4 both add a section at the end of the README. Once the first pull request of a pair is merged, the second one shows *This branch has conflicts that must be resolved.*

Your turn ★ The author of the second pull request resolves the conflict *on their branch*, with what you did in TD2: bring the new `main` into the branch, resolve, finish the merge, push. Keep *both* changes (both functions, both bullets, both sections), run the tests, and write your commands in `td3_answers.md`.

Exercise 6 — Read the team history

Where you should be: `pwd` ends with `courses/git/td3/msa-datb07-team<N>`
If not: `cd ~/Documents/courses/git/td3/msa-datb07-team<N>`

Your turn ★ Once the four pull requests are merged: `git switch main`, `git pull`, `git lg`. Find on the graph: one merge commit per pull request (message `Merge pull request #...`), and the merge commit where a conflict was resolved (message `Merge branch 'main' into ...`). Why does `git lg` show labels starting with `origin/`?

Part D — Check the protection *(10 min)*

Exercise 7 — Push on main?

Where you should be: `pwd` ends with `courses/git/td3/msa-datb07-team<N>`
If not: `cd ~/Documents/courses/git/td3/msa-datb07-team<N>`

Your turn ★ On `main`, add a line to the README, save, commit, and try `git push`. What does GitHub answer, and why?

Guided. Your commit never left your laptop: this is the exception where `git reset` is allowed. Predict, run, explain:

```
git reset --hard origin/main       # back to main as it is on GitHub
git status
git log --oneline -n 3
```

`origin/main` is `main` as it is on GitHub. `reset --hard` moves your `main` back there and throws your local commit and edits away. Never on commits that are already on GitHub: use `git revert` there.

Part E — Look at your work on GitHub *(5 min)*

1. On your team repository: *Pull requests* → *Closed*, *Issues* → *Closed*, and *Insights* → *Contributors*. Who did what, who reviewed what: GitHub keeps all of it, with names and dates.
2. **Before TD4**, if you can: tasks T1 to T4 merged, each with a review. TD4 continues in the same repository, with the same team.

Annex — If you cannot connect to GitHub

GitHub must know who you are before it accepts a push. The addresses of this TD start with `https://`: Git asks once, then remembers.

- **Windows**: at the first push, a window opens, *Sign in with your browser*. Sign in to GitHub in the browser and authorise. Nothing opens? Check `git config --global credential.helper`: it should print `manager`.
- **macOS**: the terminal asks `Username`: your GitHub user name. `Password`: **not** your GitHub password, a *token*. On GitHub: avatar → *Settings* → *Developer settings* → *Personal access tokens* → *Tokens (classic)* → *Generate new token (classic)*, tick *repo*, generate, copy it (GitHub shows it only once), paste it as the password. Nothing shows while you paste: that is normal, press `Enter`. The Keychain remembers it.
- **"Permission denied" or 403**: you are signed in, but not a collaborator of the repository: accept the invitation (Part A).
- A token is a password: never write it in a file of a repository.

Deliverable

Nothing to send: TD3 is not graded. Keep your answers to the ★ questions in `td3_answers.md` to revise for the quiz.